第7回：DB・APIを使うLLM

Function Callingで、決められた関数を呼ぶ

今日のゴールと流れ

LLMが「関数を選ぶ」ことと、Pythonが「関数を実行する」ことを区別して説明できる。
LLMが作った関数名と引数を、実行前に確かめる理由を説明できる。

内容
Function Callingの流れ、進捗DBと3つの関数
完成版 `progress-app/` を動かす・読む
穴埋め：SQLの選び方、関数の選び方
自然文で質問し、関数・引数・DBの結果と回答を照合
できないことの確認、提出

API利用時は毎回、起動するターミナルで：
`export OPENAI_API_KEY=sk-********`
終了時は停止して `unset OPENAI_API_KEY` 。

今日の進め方

段階	すること
1	`sample-app/progress-app/` の完成版を動かす
2	コードの説明を聞く
3	`session07/exercise/` の穴あきコードを埋めて動かす
4	コミットしてプッシュする
5	GitHubのURLをフォームで提出する

穴あきコードの答えは `sample-app/progress-app/` です。

文書検索と、更新されるデータ

質問	参照先	第何回
ハルシネーションとは？	資料（スライド）	第5・6回の検索
まだ終わっていない作業は？	進捗DBの `done` 列	今回
4番を完了にして	進捗DBを書き換える	今回

資料は書かれた内容を探すものでした。
進捗のように更新される状態は、DBを条件で読み、必要なら書き換えます。
今回の進捗データは、Webアプリ制作の架空の「やること」です。

Function Callingとは

1. アプリが「使える関数」の名前・説明・引数の形をLLMに渡す
2. LLMが、呼びたい関数名と引数を返す
3. Pythonが関数名と引数を確かめてから、関数を実行する
4. 実行結果をLLMへ送る
5. LLMが結果をもとに回答の文章を作る

LLMが直接SQLiteを操作するわけではありません。
LLMが作るのは「関数名と引数」だけで、実行するのはPythonです。

1問を追うための役割分担

質問：「まだ終わっていない作業は？」

担当	この例で行うこと
LLM・1回目	`get_tasks` を `status` = `todo` で呼ぶと返す
Python	関数名と引数を確かめ、決まったSQLを実行
SQLite	`done = 0` の行を返す
LLM・2回目	返った行をもとに回答文を生成

以下は処理を追う説明例です。モデルの実際の出力は実行して確認します。

進捗DBの tasks テーブル

`progress-app/progress.db` の `tasks` テーブルです。起動時に `FIRST_TASKS` の6件を入れます。

id	task	done
1	画面のHTMLとCSSを作る	1
2	FastAPIで質問を受け取るAPIを作る	1
3	SQLiteに資料を保存する	1
4	Janomeでキーワード検索を作る	0
5	回答に資料IDを付ける	0
6	GitHubにコミットしてURLを提出する	0

SQLiteに真偽値の型は無いので、`done` は1（完了）/0（未完了）で保存します。
APIでは `true` / `false` に直して返します。

INSERT OR IGNORE と初期値に戻す方法

```
cursor.executemany('INSERT OR IGNORE INTO tasks (id, task, done) VALUES (?, ?, ?)', FIRST_TASKS)
```

`INSERT OR IGNORE` は、同じidの行がすでにあれば何もしません。
再起動しても、追加した行や変更した完了フラグは上書きされません。

したいこと	操作
変更を残したまま再起動	そのまま `python main.py`
最初の6件に戻す	サーバーを止め、`progress.db` を削除してから起動

LLMが呼べる3つの関数

関数	引数	DB	すること
`get_tasks`	`status`（`all`・`todo`・`done`）	読む	進捗を調べる
`add_task`	`task`（100文字以内）	書く	やることを未完了で追加する
`set_task_done`	`task_id`（整数）、`done`（`true`・`false`）	書く	完了フラグを指定した値にする

削除する関数は用意していません。
1つの質問で呼べる関数は1回だけです。

「反対にする」ではなく「指定した値にする」

`set_task_done` は、完了フラグを `done` の値にします。

設計	同じ呼び出しが2回起きると
反対にする（切り替え）	完了にした後、未完了へ戻ってしまう
指定した値にする（今回）	完了のまま変わらない

LLMが同じ関数を2回呼んでも結果が変わらないように、引数で値を指定させます。
idは利用者が示したものだけを使い、LLMに推測させません。

完成版を動かす

```
source .venv/bin/activate
cd sample-app/progress-app
python main.py
```

ブラウザでポート8003番を開きます。DBは起動時に `init_db()` が作ります。

「まだ終わっていない作業は？」と質問します。
回答の下の「LLMが呼んだ関数とDBの結果」に、関数名・引数・行の表が出ます。
この表は `static/app.js` の `showToolResults()` が作ります。
関数を呼ばなかったときは「関数は呼ばれませんでした。」と出ます。

今回のファイル

ファイル	役割
`main.py`	質問を受け取り、`ask_with_tools()` を呼ぶ。起動時に `init_db()`
`llm.py`	`TOOLS` ・ `INSTRUCTIONS` 、 `run_tool()` 、 `ask_with_tools()`
`progress_db.py`	`init_db()` 、 `get_tasks()` 、 `add_task()` 、 `set_task_done()`
`static/app.js`	送信と、`showToolResults()` による表の表示

読み順は `main.py` の `ask()` 、 `llm.py` の `ask_with_tools()` 、 `run_tool()` 、 `progress_db.py` です。
SQLを書くのは `progress_db.py` だけです。

関数の説明を渡す

```
TOOLS = [
  {
    'type': 'function',
    'name': 'get_tasks',
    'description': 'Webアプリ制作の進捗（id・やること・完了フラグ）を調べる。'
                   'status は all（全件）・todo（未完了）・done（完了）。',
    'parameters': {...},
    'strict': True,
  },
```

`description` は、LLMが「どの関数をいつ使うか」を判断する材料です。
`add_task` の説明には「利用者が追加を頼んだときだけ使う」と書いてあります。

引数の形

```
'parameters': {
    'type': 'object',
    'properties': {'status': {'type': 'string', 'enum': ['all', 'todo', 'done']}},
    'required': ['status'],
    'additionalProperties': False,
},
'strict': True,
```

指定	意味
`enum`	値はこの選択肢だけ
`required`	必ず入れる引数
`additionalProperties: False`	それ以外の引数は入れない
`strict: True`	この形に合う引数だけを生成させる

自然文と引数の対応

質問の例	LLMが呼ぶはずの関数
まだ終わっていない作業は？	`get_tasks({"status":"todo"})`
終わった作業は？	`get_tasks({"status":"done"})`
「READMEを書く」を追加して	`add_task({"task":"READMEを書く"})`
4番を完了にして	`set_task_done({"task_id":4,"done":true})`
FastAPIとは？	関数を呼ばずに回答する場合がある

日本語の質問を、関数が受け取れる引数へ対応させるのはLLMです。
期待する引数と、実際に返った引数を画面で見比べます。

LLMへの指示

```
INSTRUCTIONS = (
    'Webアプリ制作の進捗は、必ず関数の結果だけを根拠にしてください。'
    '結果にないことは推測しないでください。'
    '進捗の追加や変更は、利用者がはっきり頼んだときだけ行ってください。'
    '複数の関数を呼ぶ必要がある場合は、1つだけ呼んでください。2つ目は呼ばなくてよいです。'
    '完了フラグを変えるときは、利用者が示したidを使い、idを推測しないでください。'
    '日本語で簡潔に回答してください。'
)
```

指示は「お願い」です。守られる保証はないので、コード側でも制限します。

最初のAPI呼び出し

```
first = client.responses.create(
    model=model,
    reasoning={'effort': 'none'},
    instructions=INSTRUCTIONS,
    input=history,
    tools=TOOLS,
    parallel_tool_calls=False, # 関数は1回に1つだけ呼ばせる
    max_output_tokens=600,
    store=False,
)
```

`tools` に関数の説明を入れます。この時点では、まだ関数を実行していません。
`history` は `[{'role': 'user', 'content': question}]` から始まります。

関数呼び出しの読み取り

```
for item in first.output:
    if item.type == 'function_call':
        rows = run_tool(item.name, item.arguments, database)
```

項目	内容
`item.name`	LLMが選んだ関数名（文字列）
`item.arguments`	引数のJSON文字列。例 `'{"status": "todo"}'`
`item.call_id`	この依頼を示すID

出力には文章など、関数以外の項目もあります。`type` で見分けます。

1回目の出力の例

主要な項目だけを示した、説明用の出力です。

```
{
  "type": "function_call",
  "name": "get_tasks",
  "arguments": "{\"status\":\"todo\"}",
  "call_id": "call_example_01"
}
```

この時点ではDBをまだ読んでいません。
`arguments` は文字列なので、Pythonで `json.loads()` して辞書にします。

実行前の検証：run_tool()

```
args = json.loads(arguments)
if not isinstance(args, dict):
    raise ValueError('引数の形が正しくありません。')

if name == 'get_tasks':
    check_keys(args, ['status'])
    return get_tasks(database, args['status'])
elif name == 'add_task':
    check_keys(args, ['task'])
    return add_task(database, args['task'])
elif name == 'set_task_done':
    check_keys(args, ['task_id', 'done'])
    return set_task_done(database, args['task_id'], args['done'])
else:
    raise ValueError('許可していない関数です。')
```

引数の名前と値の確認

```
def check_keys(args, expected_keys):
    if sorted(args.keys()) != sorted(expected_keys):
        raise ValueError('引数の形が正しくありません。')
```

確かめること	場所
関数名が3つのどれか	`run_tool()` の分岐
引数の名前がちょうど同じか	`check_keys()`
値の型、文字数、idがあるか	`progress_db.py` の各関数

関数名を `eval()` や `exec()` に渡しません。許可した3つの名前だけを `if` で選びます。

検証を通る引数・止まる引数

関数名と arguments	結果
`get_tasks` `{"status":"todo"}`	実行される
`delete_all` `{}`	許可していない関数です。
`get_tasks` `{"status":"todo","x":1}`	引数の形が正しくありません。
`get_tasks` `{"status":"unknown"}`	status は all・todo・done のどれかにしてください。
`set_task_done` `{"task_id":99,"done":true}`	id 99 のやることはありません。

`strict: True` で形を指定していても、実行するPython側で必ず確かめます。
`ValueError` は `main.py` が 400 にして、画面にその文が出ます。

SQLは決まった文だけ

```
if status == 'all':
    sql = 'SELECT id, task, done FROM tasks ORDER BY id'
elif status == 'todo':
    sql = 'SELECT id, task, done FROM tasks WHERE done = 0 ORDER BY id'
elif status == 'done':
    sql = 'SELECT id, task, done FROM tasks WHERE done = 1 ORDER BY id'
else:
    raise ValueError('status は all・todo・done のどれかにしてください。')
```

`status` の値でSQLを選ぶだけで、LLMが作った文字列をSQLに入れません。
書き込みも `INSERT INTO tasks (task, done) VALUES (?, 0)` のように、値は `?` で別に渡します。

結果を次の入力へ追加

```
history.extend(first.output)
history.append({
    'type': 'function_call_output',
    'call_id': item.call_id, # どの依頼への結果かを示す
    'output': json.dumps(rows, ensure_ascii=False),
})
```

1回目の出力	Pythonが追加する結果
type: function_call	type: function_call_output
call_id: call_example_01	call_id: call_example_01
arguments: todoの指定	output: 取得した3件のJSON文字列

関数を呼びたいという出力と、その結果の両方を `history` に入れます。

2回目のAPI呼び出し

```
final = client.responses.create(
    model=model,
    reasoning={'effort': 'none'},
    instructions=INSTRUCTIONS,
    input=history,
    max_output_tokens=600,
    store=False,
)
```

- 2回目は `tools` を渡さないので、これ以上関数は呼ばれない
- 関数を呼ばなかった質問は、1回目の回答（`first`）をそのまま使う
- 料金は2回分なので、トークン数も `first.usage` と `final.usage` を足す

実習1：関数の選び方を観察

完成版（ポート8003）で、次の4つを順に質問します。

質問	観察すること
まだ終わっていない作業は？	`get_tasks` 、 `status` は `todo` 、3件
「READMEを書く」を追加して	`add_task` 、id 7 が未完了で増える
4番を完了にして	`set_task_done` 、 `task_id` 4・ `done` true
FastAPIとは？	「関数は呼ばれませんでした。」と出るか

質問ごとに、呼ばれた関数名と引数をメモします。
最後にもう一度「まだ終わっていない作業は？」と聞き、4番が消えて7番が入ることを確かめます。

最終回答とDB結果の照合

説明用の回答例（「まだ終わっていない作業は？」）：

> 未完了は3件です。4「Janomeでキーワード検索を作る」、
> 5「回答に資料IDを付ける」、6「GitHubにコミットしてURLを提出する」です。

照合する項目	見るところ
件数	表の行数
id と やること	表の値と一字ずつ
書き足し	表にない期限や担当者を回答に足していないか

関数を正しく呼べていても、回答の確認は別に必要です。

実習2：get_tasks() のSQLを埋める

`session07/exercise/progress_db.py` の `get_tasks()` を完成させます。

1. `TODO` を探す（`status` に応じてSQLを選ぶ場所）
2. `if` / `elif` / `else` の8行のコメントを外す
3. `raise ValueError('TODO: ...')` の行を消す
4. `cd session07/exercise` で `python` を起動して確かめる

```
from progress_db import init_db, get_tasks
init_db('progress.db')
get_tasks('progress.db', 'todo')
```

埋める前は `ValueError`（TODOの文）で止まります。
埋めた後は id 4・5・6 の3件の辞書のリストが返ります。`all` ・ `done` も試します。

実習3：run_tool() を埋める

`session07/exercise/llm.py` の `run_tool()` を完成させます。

1. `TODO` を探す（関数名ごとの分岐）
2. `if` / `elif` / `else` のコメントを外す。`check_keys` の行も含めて11行
3. `raise ValueError('TODO: ...')` の行を消す
4. 同じフォルダの `python` で確かめる（APIキーは不要）

```
from llm import run_tool
run_tool('get_tasks', '{"status": "todo"}', 'progress.db')
run_tool('delete_all', '{}', 'progress.db')
run_tool('get_tasks', '{"status": "todo", "x": 1}', 'progress.db')
```

1つ目は3件、2つ目は「許可していない関数です。」、3つ目は「引数の形が正しくありません。」です。

実習4：自然文で質問して照合

完成版のサーバーを止めてから、exercise 側を起動します。

```
cd session07/exercise
python main.py
```

ポート8003番を開き、実習1と同じ4つの質問を送ります。

見比べるもの	期待すること
関数名・引数	実習1でメモしたものと同じ
DBの結果の表	実習2で `python` から見た行と同じ
回答	表の件数・id・やることと一致

穴が残っていると、画面に `TODO: …` の文が出ます。

実習5：できないことを確かめる

質問	期待すること
全部削除して	削除する関数が無いので、消えない
3番を未完了に戻して	`set_task_done` で `done` false
5番と6番を完了にして	関数は1つだけ。片方しか変わらない

削除は、LLMが「削除したい」と言っても関数そのものがありません。
5番と6番は、`parallel_tool_calls=False` と指示により1回しか呼べないことを確かめます。
残った方は「6番を完了にして」のように、もう1問で頼みます。

権限をコードで制限

制限	場所
関数名を3つに固定	`run_tool()` の `if` と `else`
引数の名前をちょうど同じに	`check_keys()`
型・文字数・idの存在	`progress_db.py` の各関数
SQLは決まった文、値は `?`	`progress_db.py`
削除関数を作らない	`TOOLS`
2回目に `tools` を渡さない	`ask_with_tools()`

`eval()` 、 `exec()` 、LLMが生成したSQLは実行しません。
指示文に「安全に」と書くだけでなく、アプリが実行できる操作そのものを狭めます。

失敗した場所の切り分け

観察した状態	まず確認すること
importで止まる	仮想環境と依存パッケージ
画面に `TODO: ...` の文が出る	穴埋めと仮の `raise` 、どちらのファイルか
「許可していない関数です。」	`run_tool()` の関数名の分岐
「引数の形が正しくありません。」	`check_keys()` に渡す引数名のリスト
todo のはずが done になった	質問の文、関数の説明、実際の引数
DBの結果は正しいが回答が違う	2回目の入力と生成結果
「API呼び出しに失敗しました」	キー・モデル・利用枠

相談時は、質問の文と画面の関数名・引数を伝えます。キーは共有しません。

確認問題

1. 関数を実際に実行するのは、LLMとPythonのどちら？
2. `strict: True` で引数の形を指定したら、Pythonの検証は不要？
3. `call_id` は何と何を対応させる？
4. 完了フラグを「反対にする」ではなく「指定した値にする」関数にしたのはなぜ？

提出物と次回

`session07/exercise/` の2つのファイル（ `progress_db.py` 、 `llm.py` ）をコミットし、プッシュします。
`progress.db` は `.gitignore` にあるので、コミットされません。

```
cd ../..  # リポジトリのルートへ戻る
git add session07/exercise
git commit -m "第7回: Function Calling"
git push
```

GitHubで `session07/exercise` を開き、そのURLをフォームから提出します。

次回は、完成したアプリを評価し、安全な運用を点検して、制作の企画に入ります。